6
EXTENDING BURP PROXY



If you've ever tried hacking a web application, you've likely used Burp Suite to perform spidering, proxy browser traffic, and carry out other attacks. Burp Suite also allows you to create your own tooling, called *extensions*. Using Python, Ruby, or pure Java, you can add panels in the Burp GUI and build automation techniques into Burp Suite. We'll take advantage of this feature to write some handy tooling for performing attacks and extended reconnaissance. The first extension will use an intercepted HTTP request from Burp Proxy as a seed for a mutation fuzzer that runs in Burp Intruder. The second extension will communicate with the Microsoft Bing API to show us all virtual hosts located on the same IP address as a target site, as well as any subdomains detected for the target domain. Finally, we'll build an extension to create a word list from a target website that you can use in a brute-force password attack.

This chapter assumes that you've played with Burp before and know how to trap requests with the Proxy tool, as well as how to send a trapped request to Burp Intruder. If you need a tutorial on how to do these tasks, visit PortSwigger Web Security (*http://www.portswigger.net/*) to get started.

We have to admit that when we first started exploring the Burp Extender API, it took us some time to understand how it worked. We found it a bit confusing, as we're pure Python guys and have limited Java development experience. But we found a number of extensions on the Burp website that taught us how other folks had developed extensions. We used that prior art to help us understand how to begin implementing our own code. This chapter will cover some basics on extending functionality, but we'll also show you how to use the API documentation as a guide.

Setting Up

Burp Suite comes installed by default on Kali Linux. If you're using a different machine, download Burp from *http://www.portswigger.net/* and set it up.

As sad as it makes us to admit this, you'll require a modern Java installation. Kali Linux has one installed. If you're on a different platform, use your system's installation method (such as apt, yum, or rpm) to get one. Next, install *Jython*, a Python 2 implementation written in Java. Up until now, all of our code has used Python 3 syntax, but in this chapter we'll revert to Python 2, since that's what Jython expects. You can find this JAR file on the on the official site, *https://www.jython.org/download.html*. Select the Jython 2.7 Standalone Installer. Save the JAR file to an easy-to-remember location, such as your Desktop.

Next, either double-click the Burp icon on your Kali machine or run Burp from the command line:

```
#> java -XX:MaxPermSize=1G -jar burpsuite_pro_v1.6.jar
```

This will fire up Burp, and you should see its graphical user interface (GUI) full of wonderful tabs, as shown in *Figure 6-1*.

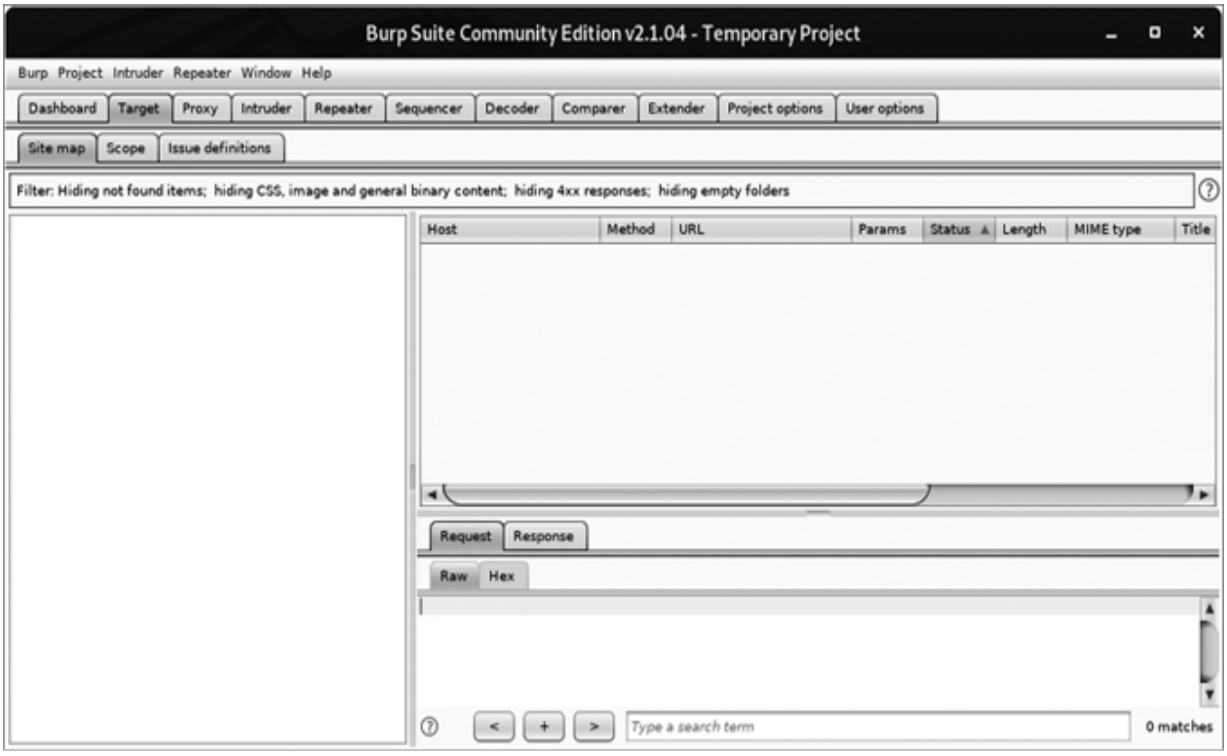


Figure 6-1: Burp Suite GUI loaded properly

Now let's point Burp at our Jython interpreter. Click the **Extender** tab and then click the **Options** tab. In the Python Environment section, select the location of your Jython JAR file, as shown in *Figure 6-2*. You can leave the rest of the options alone. We're ready to start coding our first extension. Let's get rocking!

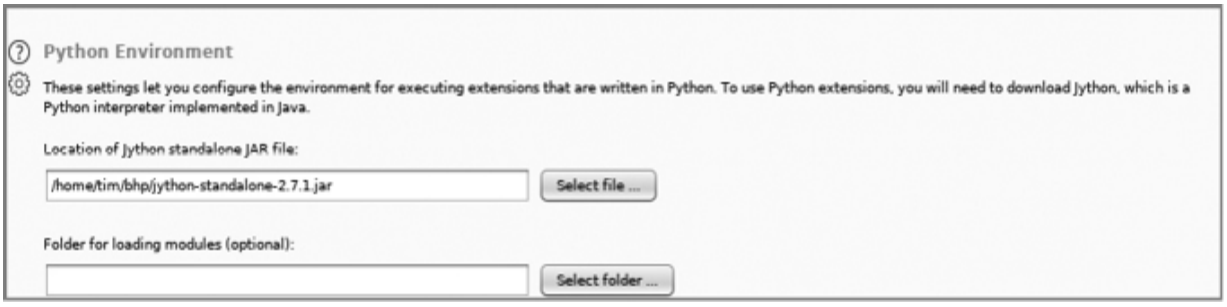


Figure 6-2: Configuring the Jython interpreter location

Burp Fuzzing

At some point in your career, you may find yourself attacking a web application or service that doesn't allow you to use traditional web application assessment tools. For example, the application might use too many parameters, or it may be obfuscated in some way that makes performing a manual test far too time-consuming. We've been guilty of running standard tools that can't deal with strange protocols, or even JSON in a lot of cases. This is where you'll find it useful to establish a solid baseline of HTTP traffic, including authentication cookies, while passing off the body of the request to a custom fuzzer. This fuzzer can then manipulate the payload in any way you choose. We'll work on our first Burp extension by creating the world's simplest web application fuzzer, which you can then expand into something more intelligent.

Burp has a number of tools you can use when you're performing web application tests. Typically, you'll trap all requests using the Proxy, and when you see an interesting one, you'll send it to another Burp tool. A common technique is to send them to the Repeater tool, which lets you replay web traffic as well as manually modify any interesting spots. To perform more automated attacks in query parameters, you can send a request to the Intruder tool, which attempts to automatically figure out which areas of the web traffic you should modify and then allows you to use a variety of attacks to try to elicit error messages or tease out vulnerabilities. A Burp extension can interact in numerous ways with the Burp suite of tools. In our case, we'll bolt additional functionality directly onto the Intruder tool.

Our first instinct is to take a look at the Burp API documentation to determine what Burp classes we need to extend in order to write our custom extension. You can access this documentation by clicking the **Extender** tab and then clicking the **APIs** tab. The API can look a little daunting because it's very Java-y. But notice that the Burp developers have aptly named each class, making it easy to figure out where we want to start. In particular, because we're trying to fuzz web requests during an Intruder attack, we might want to focus on

the `IIntruderPayloadGeneratorFactory` and `IIntruderPayloadGenerator` classes. Let's take a look at what the documentation says for the `IIntruderPayloadGeneratorFactory` class:

```
/**
 * Extensions can implement this interface and then call
1 *
IBurpExtenderCallbacks.registerIntruderPayloadGeneratorFactor
y()
 * to register a factory for custom Intruder payloads.
*/

public interface IIntruderPayloadGeneratorFactory
{
    /**
     * This method is used by Burp to obtain the name of the
payload
     * generator. This will be displayed as an option within
the
     * Intruder UI when the user selects to use extension-
generated
     * payloads.

     *
     * @return The name of the payload generator.
     */
2 String getGeneratorName();

    /**
     * This method is used by Burp when the user starts an
Intruder
     * attack that uses this payload generator.

     * @param attack
     * An IIntruderAttack object that can be queried to
obtain details
     * about the attack in which the payload generator will
be used.

     * @return A new instance of
     * IIntruderPayloadGenerator that will be used to
generate
     * payloads for the attack.
     */
```

```
3 IIntruderPayloadGenerator createNewInstance(IIntruderAttack
  attack);
  }
```

The first bit of documentation 1 tells how to correctly register our extension with Burp. We'll extend the main Burp class as well as the `IIntruderPayloadGeneratorFactory` class. Next, we see that Burp expects two methods in our main class. Burp will call the `getGeneratorName` method 2 to retrieve the name of our extension, and we're expected to return a string. The `createNewInstance` method 3 expects us to return an instance of the `IIntruderPayloadGenerator`, a second class we'll have to create.

Now let's implement the actual Python code to meet these requirements. Then we'll figure out how to add the `IIntruderPayloadGenerator` class. Open a new Python file, name it *bhp_fuzzer.py*, and punch out the following code:

```
1 from burp import IBurpExtender
  from burp import IIntruderPayloadGeneratorFactory
  from burp import IIntruderPayloadGenerator

  from java.util import List, ArrayList

  import random

2 class BurpExtender(IBurpExtender,
  IIntruderPayloadGeneratorFactory):
      def registerExtenderCallbacks(self, callbacks):
          self._callbacks = callbacks
          self._helpers   = callbacks.getHelpers()

       3
  callbacks.registerIntruderPayloadGeneratorFactory(self)

          return

   4 def getGeneratorName(self):
          return "BHP Payload Generator"

   5 def createNewInstance(self, attack):
          return BHPFuzzer(self, attack)
```

This simple skeleton outlines what we need in order to satisfy the first set of requirements. We have to first import the `IBurpExtender` class 1, a requirement for every extension we write. We follow this up by importing the classes necessary for creating an Intruder payload generator. Next, we define the `BurpExtender` class 2, which extends the `IBurpExtender` and `IIntruderPayloadGeneratorFactory` classes. We then use the `registerIntruderPayloadGeneratorFactory` method 3 to register our class so that the Intruder tool is aware that we can generate payloads. Next, we implement the `getGeneratorName` method 4 to simply return the name of our payload generator. Finally, we implement the `createNewInstance` method 5, which receives the attack parameter and returns an instance of the `IIntruderPayloadGenerator` class, which we called `BHPFuzzer`.

Let's have a peek at the documentation for the `IIntruderPayloadGenerator` class so we know what to implement:

```
/**
 * This interface is used for custom Intruder payload
generators.
 * Extensions
 * that have registered an
 * IIntruderPayloadGeneratorFactory must return a new
instance of
 * this interface when required as part of a new Intruder
attack.
 */

public interface IIntruderPayloadGenerator
{
 /**
 * This method is used by Burp to determine whether the
payload
 * generator is able to provide any further payloads.
 *
 * @return Extensions should return
 * false when all the available payloads have been used up,
 * otherwise true
 */
1 boolean hasMorePayloads();

 /**
```

```
     * This method is used by Burp to obtain the value of the
    next payload.
     *
     * @param baseValue The base value of the current payload
    position.
     * This value may be null if the concept of a base value is
    not
     * applicable (e.g. in a battering ram attack).
     * @return The next payload to use in the attack.
     */
2   byte[] getNextPayload(byte[] baseValue);

     /**
     * This method is used by Burp to reset the state of the
    payload
     * generator so that the next call to
     * getNextPayload() returns the first payload again. This
     * method will be invoked when an attack uses the same
    payload
     * generator for more than one payload position, for example
    in a
     * sniper attack.
     */
3   void reset();
    }
```

Okay! Now we know we need to implement the base class, which needs to expose three methods. The first method, `hasMorePayloads`1, is there to decide whether to continue sending mutated requests back to Burp Intruder. We'll use a counter to deal with this. Once the counter reaches the maximum level, we'll return `False` to stop generating fuzzing cases. The `getNextPayload` method 2 will receive the original payload from the HTTP request that you trapped. Alternatively, if you selected multiple payload areas in the HTTP request, you'll receive only the bytes you plan to fuzz (more on this later). This method allows us to fuzz the original test case and then return it for Burp to send. The last method, `reset`3, is there so that if we generate a known set of fuzzed requests, the fuzzer can iterate through those values for each payload position designated in the Intruder tab. Our fuzzer isn't so fussy; it will always just keep randomly fuzzing each HTTP request.

Now let's see how this looks when we implement it in Python. Add the following code to the bottom of *bhp_fuzzer.py*:

```
1 class BHPFuzzer(IIntruderPayloadGenerator):
      def __init__(self, extender, attack):
          self._extender = extender
          self._helpers  = extender._helpers
          self._attack   = attack
       2 self.max_payloads   = 10
          self.num_iterations = 0

          return


    3 def hasMorePayloads(self):
          if self.num_iterations == self.max_payloads:
              return False
          else:
              return True

    4 def getNextPayload(self,current_payload):
          # convert into a string
       5 payload = "".join(chr(x) for x in current_payload)

          # call our simple mutator to fuzz the POST
       6 payload = self.mutate_payload(payload)

          # increase the number of fuzzing attempts
       7 self.num_iterations += 1

          return payload

      def reset(self):
          self.num_iterations = 0
          return
```

We start by defining a `BHPFuzzer` class 1 that extends the class `IIntruderPayloadGenerator` class. We define the required class variables and then add the `max_payloads`2 and `num_iterations` variables used to let Burp know when we've finished fuzzing. You could, of course, let the extension run forever if you'd like, but for testing purposes, we'll set time limits. Next, we implement the

`hasMorePayloads` method 3, which simply checks whether we've reached the maximum number of fuzzing iterations. You could modify this to continually run the extension by always returning `True`. The `getNextPayload` method 4 receives the original HTTP payload, and it's here that we'll be fuzzing. The `current_payload` variable arrives as a byte array, so we convert this to a string 5 and then pass it to the `mutate_payload` fuzzing method 6. We then increment the `num_iterations` variable 7 and return the mutated payload. Our last method is the `reset` method, which returns without doing anything.

Now let's write the world's simplest fuzzing method, which you can modify to your heart's content. For instance, this method knows the value of the current payload, so if you have a tricky protocol that needs something special, like a CRC checksum or a length field, you could perform those calculations inside this method before returning. Add the following code to *bhp_fuzzer.py*, inside the `BHPFuzzer` class:

```
    def mutate_payload(self,original_payload):
        # pick a simple mutator or even call an external
script
        picker = random.randint(1,3)

        # select a random offset in the payload to mutate
        offset  = random.randint(0,len(original_payload)-1)

     1 front, back = original_payload[:offset],
original_payload[offset:]

        # random offset insert a SQL injection attempt
        if picker == 1:
         2 front += "'"

            # jam an XSS attempt in
        elif picker == 2:
         3 front += "<script>alert('BHP!');</script>"

        # repeat a random chunk of the original payload
        elif picker == 3:
         4 chunk_length = random.randint(0, len(back)-1)
            repeater = random.randint(1, 10)
            for _ in range(repeater):
                front += original_payload[:offset +
```

```
chunk_length]

    5 return front + back
```

First, we take the payload and split it into two random-length chunks, `front` and `back`1. Then, we randomly pick from three mutators: a simple SQL injection test that adds a single-quote to the end of the `front` chunk 2, a cross-site scripting (XSS) test that adds a script tag to the end of the `front` chunk 3, and a mutator that selects a random chunk from the original payload, repeats it a random number of times, and adds the result to the end of the `front` chunk 4. Then, we add the `back` chunk to the altered `front` chunk to complete the mutated payload 5. We now have a Burp Intruder extension we can use. Let's take a look at how to load it.

Kicking the Tires

First, we have to load the extension and make sure it contains no errors. Click the **Extender** tab in Burp and then click the **Add** button. A screen should appear, allowing you to point Burp at the fuzzer. Ensure that you set the same options as the ones shown in *Figure 6-3*.

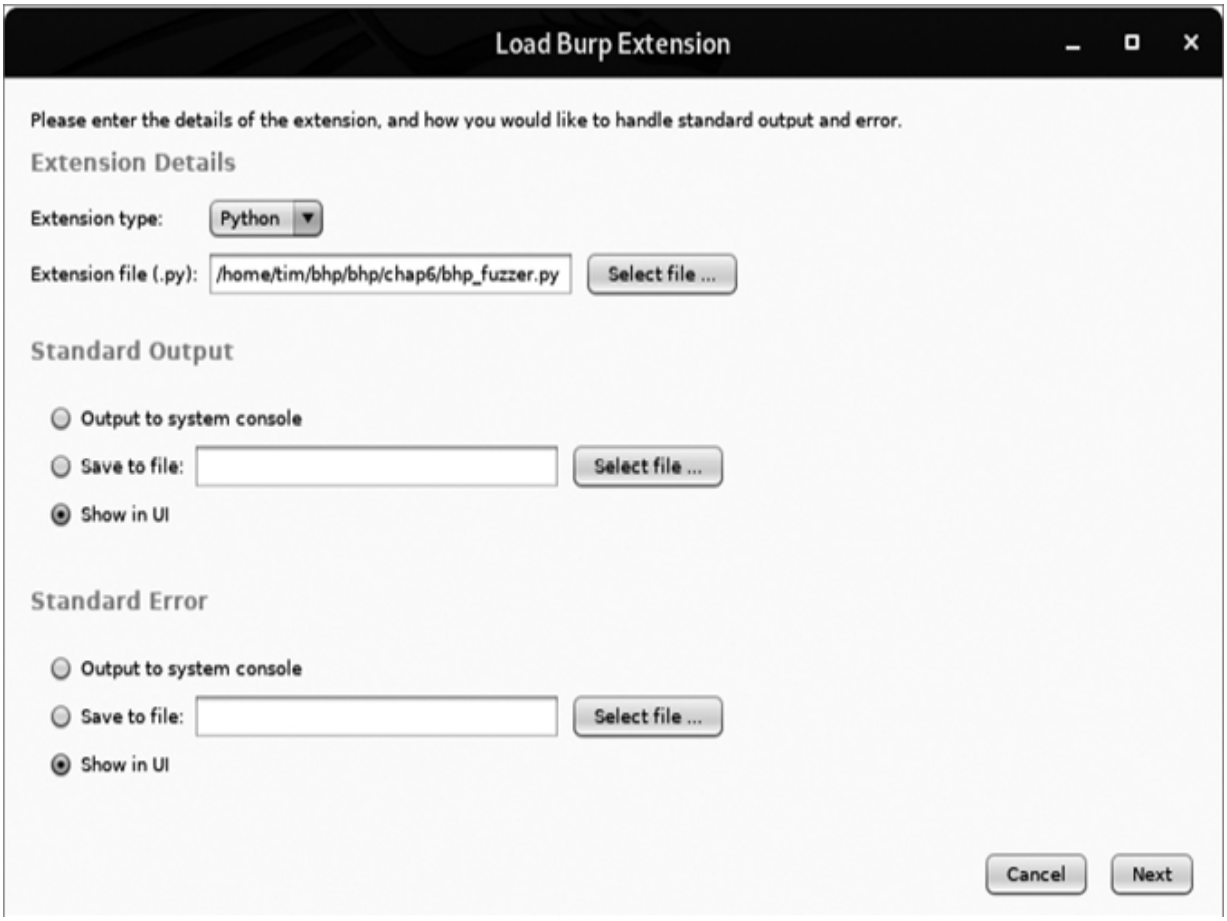


Figure 6-3: Setting Burp to load our extension

Click **Next**, and Burp should begin loading the extension. If there are errors, click the **Errors** tab, debug any typos, and then click **Close**. Your Extender screen should now look like *Figure 6-4*.

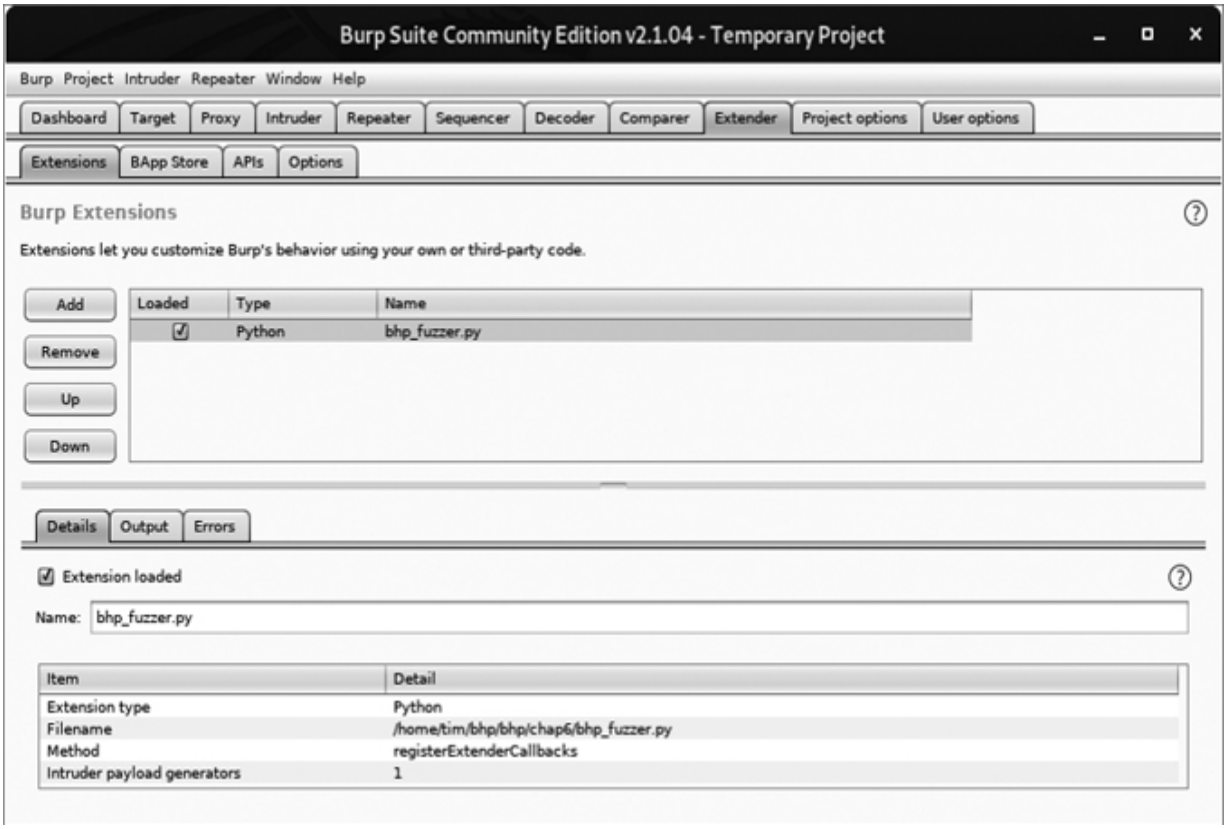


Figure 6-4: Burp Extender showing that our extension is loaded

As you can see, our extension has loaded and Burp has identified the registered Intruder payload generator. We're now ready to leverage the extension in a real attack. Make sure your web browser is set to use Burp Proxy as a localhost proxy on port 8080. Now let's attack the same Acunetix web application from Chapter 5. Simply browse to *http://testphp.vulnweb.com/.*

As an example, the authors used the little search bar on their site to submit a search for the string `"test"`. *Figure 6-5* shows how you can see this request in the HTTP history tab of the Proxy menu. Right-click the request to send it to Intruder.

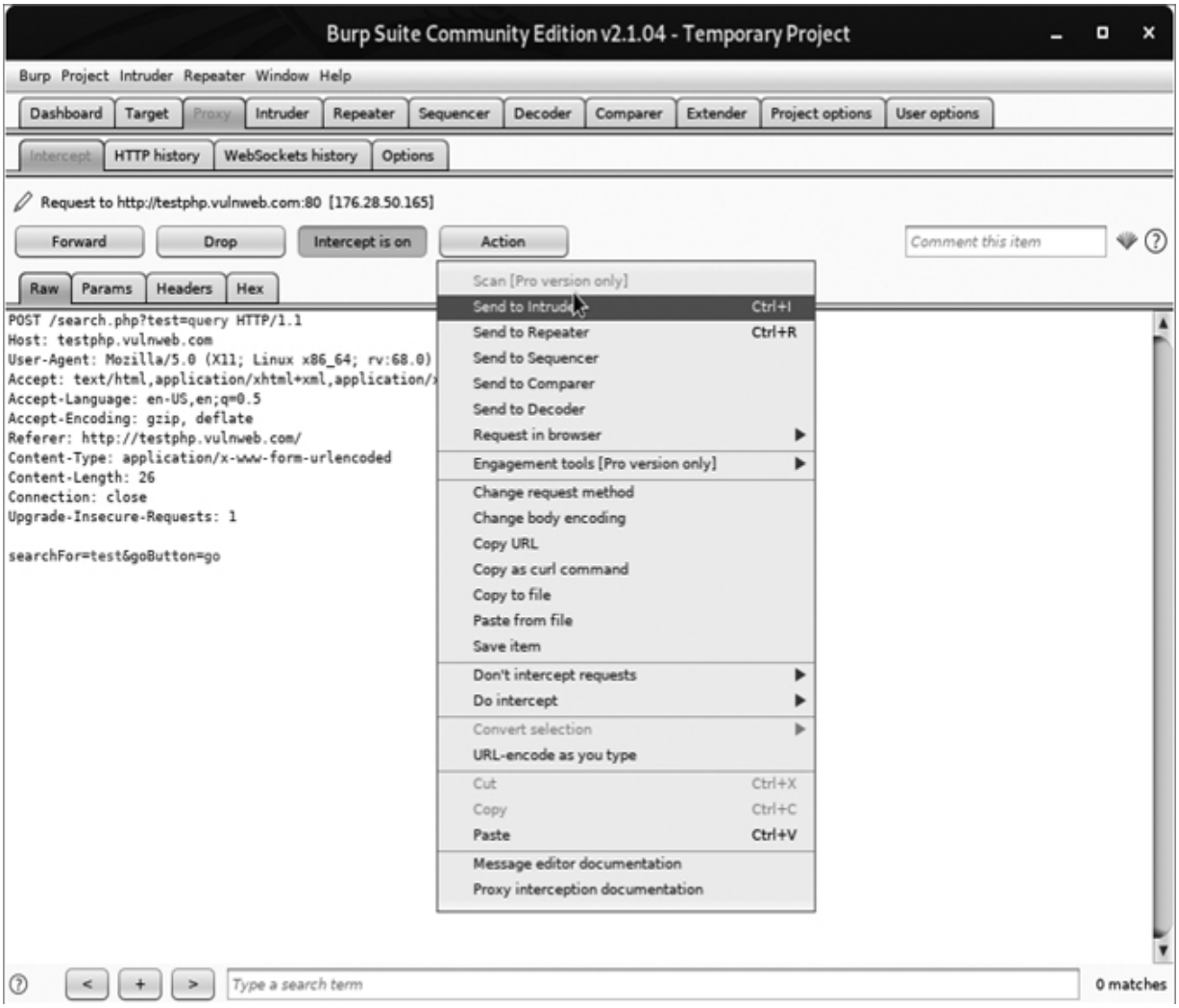


Figure 6-5: Selecting an HTTP request to send to Intruder

Now switch to the **Intruder** tab and click the **Positions** tab. A screen should appear, showing each query parameter highlighted. This is Burp's way of identifying the spots we should be fuzzing. You can try moving the payload delimiters around or selecting the entire payload to fuzz if you choose, but for now, let's let Burp decide what to fuzz. For clarity, see *Figure 6-6*, which shows how payload highlighting works.

Now click the **Payloads** tab. In this screen, click the **Payload type** drop-down and select **Extension-generated**. In the Payload Options section, click the **Select generator** button and choose **BHP Payload**

Generator from the drop-down. Your Payload screen should now look like *Figure 6-7*.

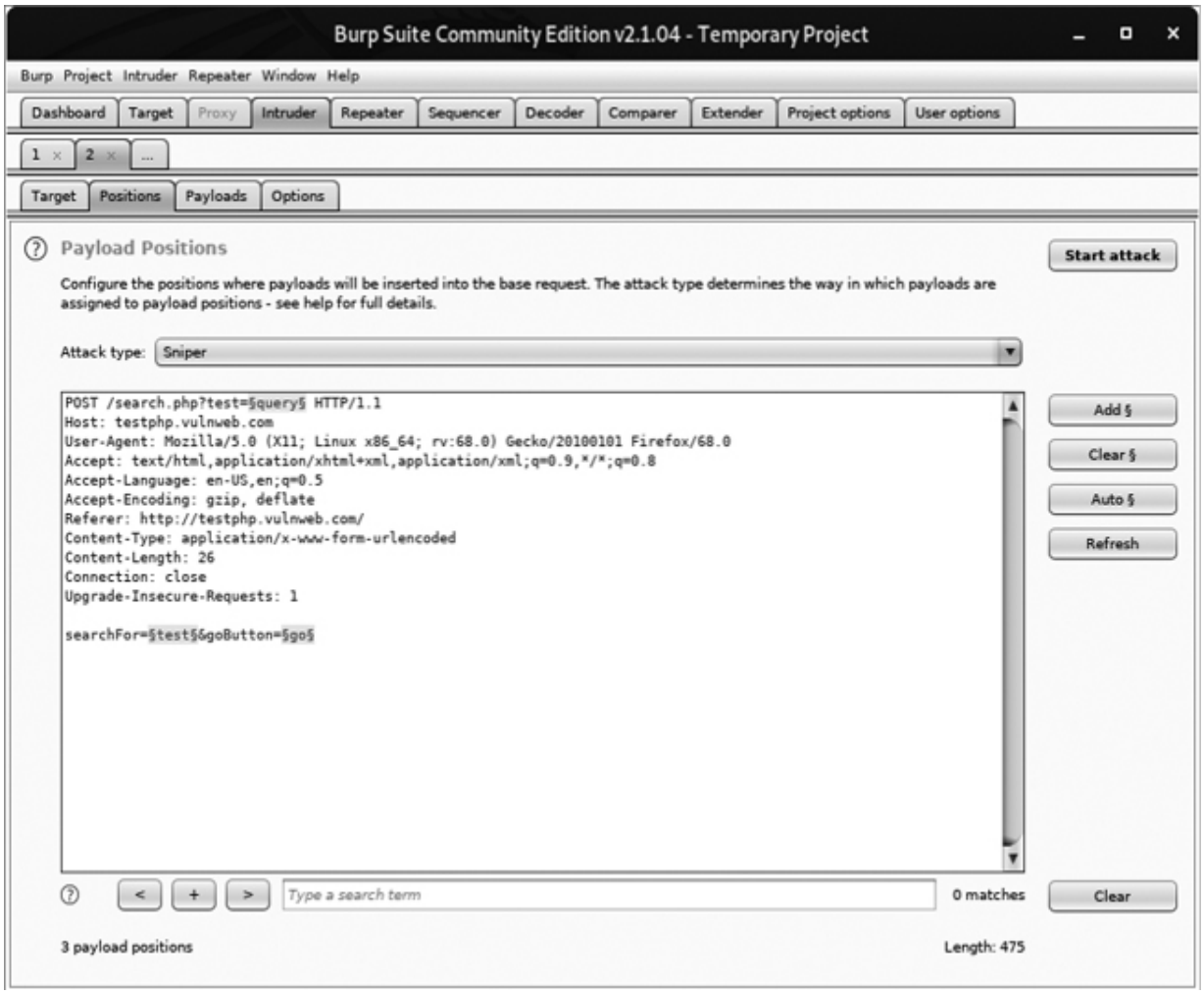


Figure 6-6: Burp Intruder highlighting payload parameters

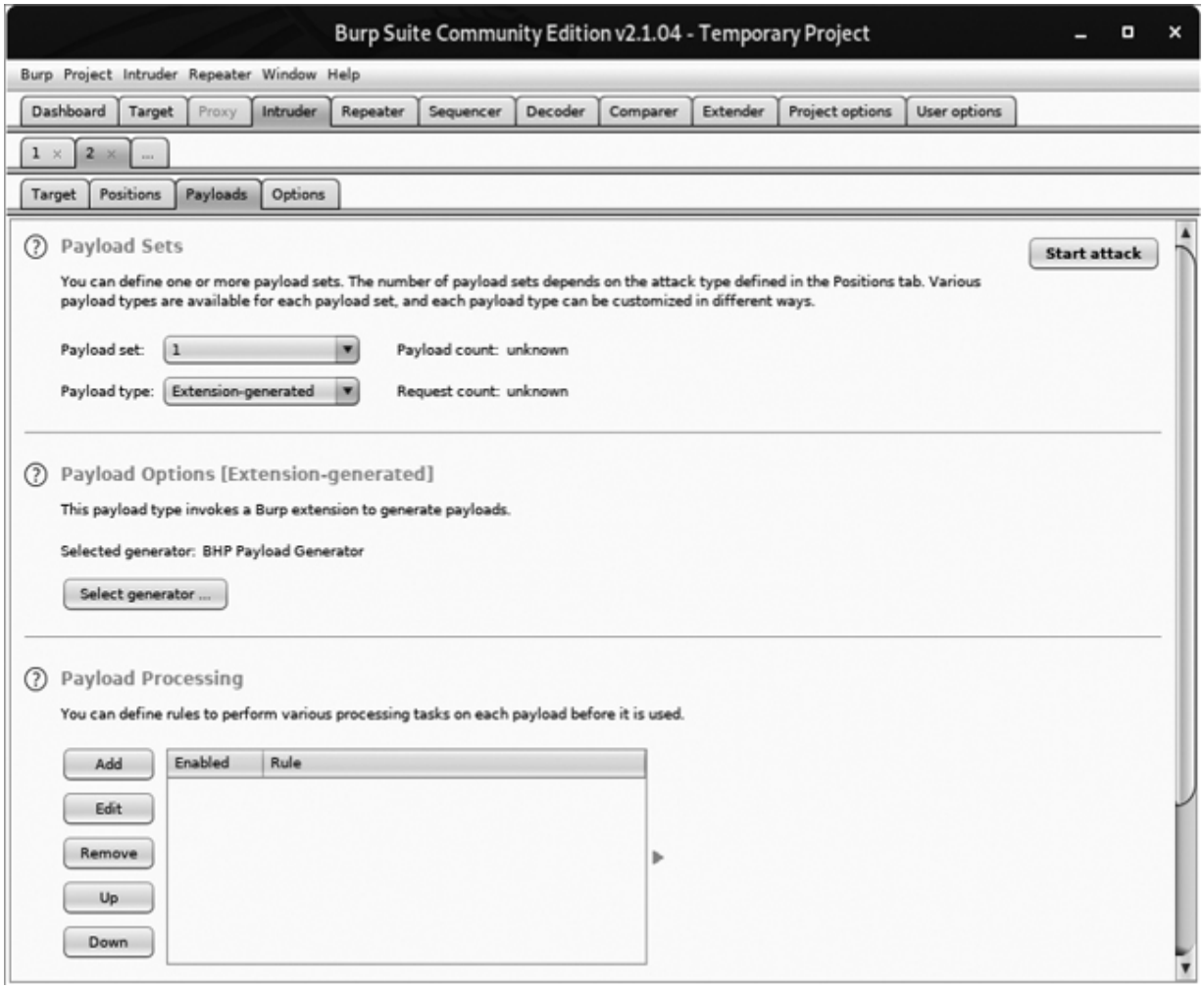


Figure 6-7: Using our fuzzing extension as a payload generator

Now we're ready to send requests. At the top of the Burp menu bar, click **Intruder** and then select **Start Attack**. Burp should begin sending fuzzed requests, and soon you'll be able to quickly go through the results. When the authors ran the fuzzer, we received the output shown in *Figure 6-8*.

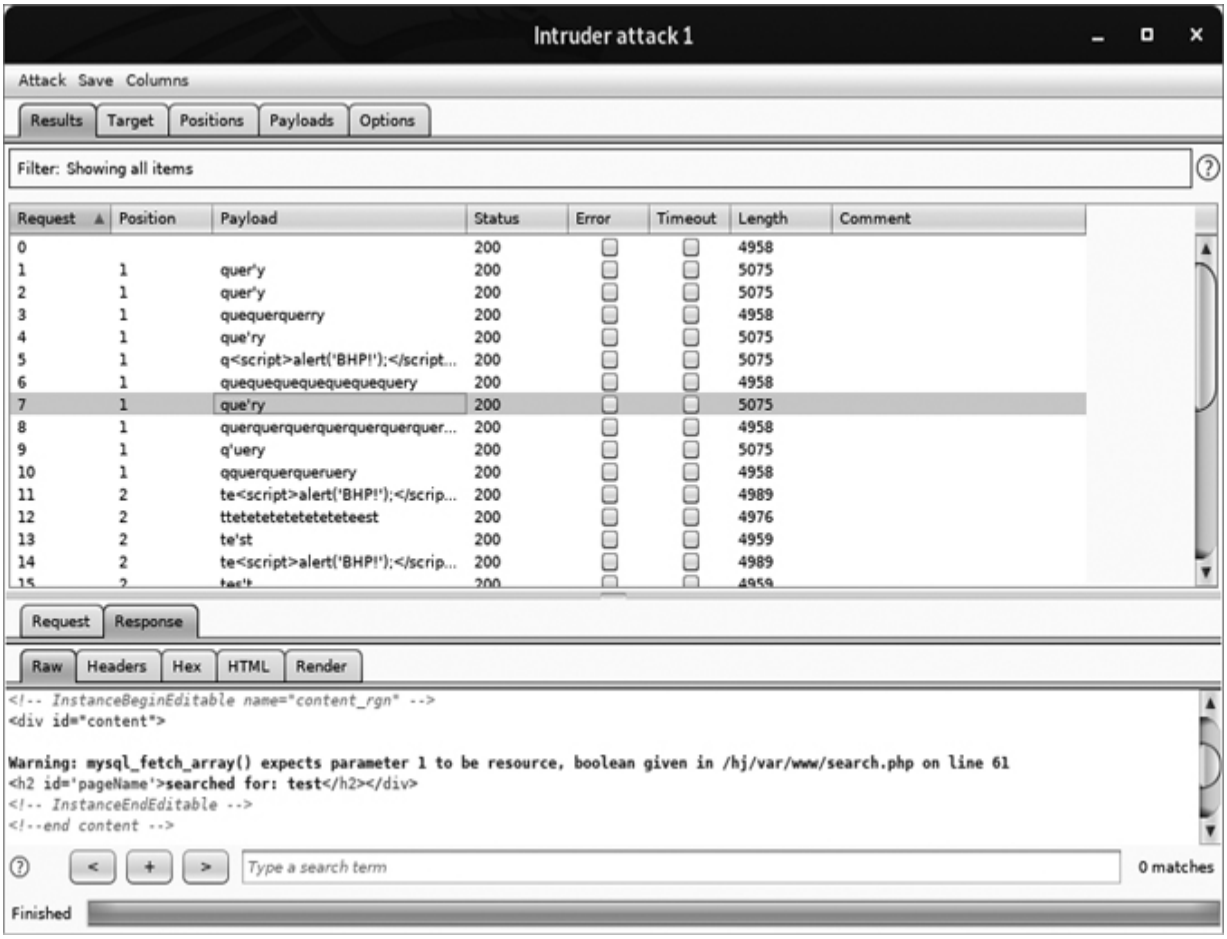


Figure 6-8: Our fuzzer running in an Intruder attack

As you can see from the bold warning in the response to request 7, we've discovered what appears to be a SQL injection vulnerability.

Even though we built this fuzzer for demonstration purposes only, you'll be surprised how effective it can be for getting a web application to output errors, disclose application paths, or generate behavior that lots of other scanners might miss. Most importantly, we managed to get our custom extension to work with Burp's Intruder attacks. Now let's create an extension that will help us perform extended reconnaissance against a web server.

Using Bing for Burp

It's not uncommon for a single web server to serve several web applications, some of which you might not be aware of. If you're attacking the server, you should do your best to discover these other hostnames, because they might give you an easier way to get a shell. It's not rare to find an insecure web application, or even development resources, located on the same machine as your target. Microsoft's Bing search engine has search capabilities that allow you to query Bing for all websites it finds on a single IP address using the "IP" search modifier. Bing will also tell you all of the subdomains of a given domain if you use the "domain" search modifier.

Now, we could use a scraper to submit these queries to Bing and then get the HTML in the results, but that would be bad manners (and also violate most search engines' terms of use). In order to stay out of trouble, we'll instead use the Bing API to submit these queries programmatically and parse the results ourselves. (Visit *https://www.microsoft.com/en-us/bing/apis/bing-web-search-api/* to get set up with your own free Bing API key.) Except for a context menu, we won't implement any fancy Burp GUI additions with this extension; we'll simply output the results into Burp each time we run a query, and any detected URLs to Burp's target scope will be added automatically.

Because we already walked you through how to read the Burp API documentation and translate it into Python, let's get right to the code. Crack open *bhp_bing.py* and hammer out the following:

```
from burp import IBurpExtender
from burp import IContextMenuFactory

from java.net import URL
from java.util import ArrayList
from javax.swing import JMenuItem
from thread import start_new_thread

import json
import socket
import urllib
API_KEY = "YOURKEY"
API_HOST = 'api.cognitive.microsoft.com'
```

```
2 class BurpExtender(IBurpExtender, IContextMenuFactory):
      def registerExtenderCallbacks(self, callbacks):
          self._callbacks = callbacks
          self._helpers   = callbacks.getHelpers()
          self.context    = None

          # we set up our extension
          callbacks.setExtensionName("BHP Bing")
       3  callbacks.registerContextMenuFactory(self)

          return

      def createMenuItems(self, context_menu):
          self.context = context_menu
          menu_list = ArrayList()
       4 menu_list.add(JMenuItem(
              "Send to Bing", actionPerformed=self.bing_menu))
          return menu_list
```

This is the first bit of our Bing extension. Make sure you paste your Bing API key in place 1. You're allowed 1,000 free searches per month. We begin by defining a `BurpExtender` class 2 that implements the standard `IBurpExtender` interface, and the `IContextMenuFactory`, which allows us to provide a context menu when a user right-clicks a request in Burp. This menu will display a "Send to Bing" selection. We register a menu handler 3 that will determine which site the user clicked, enabling us to construct our Bing queries. Then we set up a `createMenuItem` method, which will receive an `IContextMenuInvocation` object and use it to determine which HTTP request the user selected. The last step is to render the menu item and handle the click event with the `bing_menu` method 4.

Now let's perform the Bing query, output the results, and add any discovered virtual hosts to Burp's target scope:

```
    def bing_menu(self,event):

        # grab the details of what the user clicked
     1  http_traffic = self.context.getSelectedMessages()

        print("%d requests highlighted" % len(http_traffic))
```

```
        for traffic in http_traffic:
            http_service = traffic.getHttpService()
            host         = http_service.getHost()

            print("User selected host: %s" % host)
            self.bing_search(host)

        return


    def bing_search(self,host):
        # check if we have an IP or hostname
        try:
         2 is_ip = bool(socket.inet_aton(host))
        except socket.error:
            is_ip = False

        if is_ip:
            ip_address = host
            domain = False
        else:
            ip_address = socket.gethostbyname(host)
            domain = True

     3 start_new_thread(self.bing_query, ('ip:%s' %
ip_address,))

        if domain:
         4 start_new_thread(self.bing_query, ('domain:%s' %
host,))
```

The `bing_menu` method gets triggered when the user clicks the context menu item we defined. We retrieve the highlighted HTTP requests 1. Then we retrieve the host portion of each request and send it to the `bing_search` method for further processing. The `bing_search` method first determines if the host portion is an IP address or a hostname 2. We then query Bing for all virtual hosts that have the same IP address 3 as the host. If our extension received a domain as well, then we do a secondary search for any subdomains that Bing may have indexed 4.

Now let's install the plumbing we'll need in order to send the request to Bing and parse the results using Burp's HTTP API. Add the following code within the `BurpExtender` class:

```
    def bing_query(self,bing_query_string):
        print('Performing Bing search: %s' %
bing_query_string)
        http_request = 'GET https://%s/bing/v7.0/search?' %
API_HOST
        # encode our query
        http_request += 'q=%s HTTP/1.1\r\n' %
urllib.quote(bing_query_string)
        http_request += 'Host: %s\r\n' % API_HOST
        http_request += 'Connection:close\r\n'
    1 http_request += 'Ocp-Apim-Subscription-Key: %s\r\n' %
API_KEY
        http_request += 'User-Agent: Black Hat
Python\r\n\r\n'

    2 json_body = self._callbacks.makeHttpRequest(
            API_HOST, 443, True, http_request).tostring()
    3 json_body = json_body.split('\r\n\r\n', 1)[1]

        try:
        4 response = json.loads(json_body)
        except (TypeError, ValueError) as err:
            print('No results from Bing: %s' % err)
        else:
            sites = list()
            if response.get('webPages'):
                sites = response['webPages']['value']
            if len(sites):
                for site in sites:
                5 print('*'*100)
                    print('Name: %s       ' % site['name'])
                    print('URL: %s        ' % site['url'])
                    print('Description: %r' %
site['snippet'])
                    print('*'*100)

                    java_url = URL(site['url'])
                6 if not
self._callbacks.isInScope(java_url):
                        print('Adding %s to Burp scope' %
```

```
site['url'])

self._callbacks.includeInScope(java_url)
                    else:
                        print('Empty response from Bing.: %s'
                                % bing_query_string)
        return
```

Burp's HTTP API requires that we build the entire HTTP request as a string before sending it. We also need to add our Bing API key to make the API call1. We then send the HTTP request 2 to the Microsoft servers. When the response returns, we split off the headers 3 and then pass it to our JSON parser 4. For each set of results, we output some information about the site that we discovered 5. If the discovered site isn't in Burp's target scope 6, we automatically add it.

In doing so, we've blended the Jython API and pure Python in a Burp extension. This should help us do additional recon work when we're attacking a particular target. Let's take it for a spin.

Kicking the Tires

To get the Bing search extension working, use the same procedure we used for the fuzzing extension. When it's loaded, browse to *http://testphp.vulnweb.com/* and then right-click the GET request you just issued. If the extension loads properly, you should see the menu option Send to Bing displayed, as shown in *Figure 6-9*.

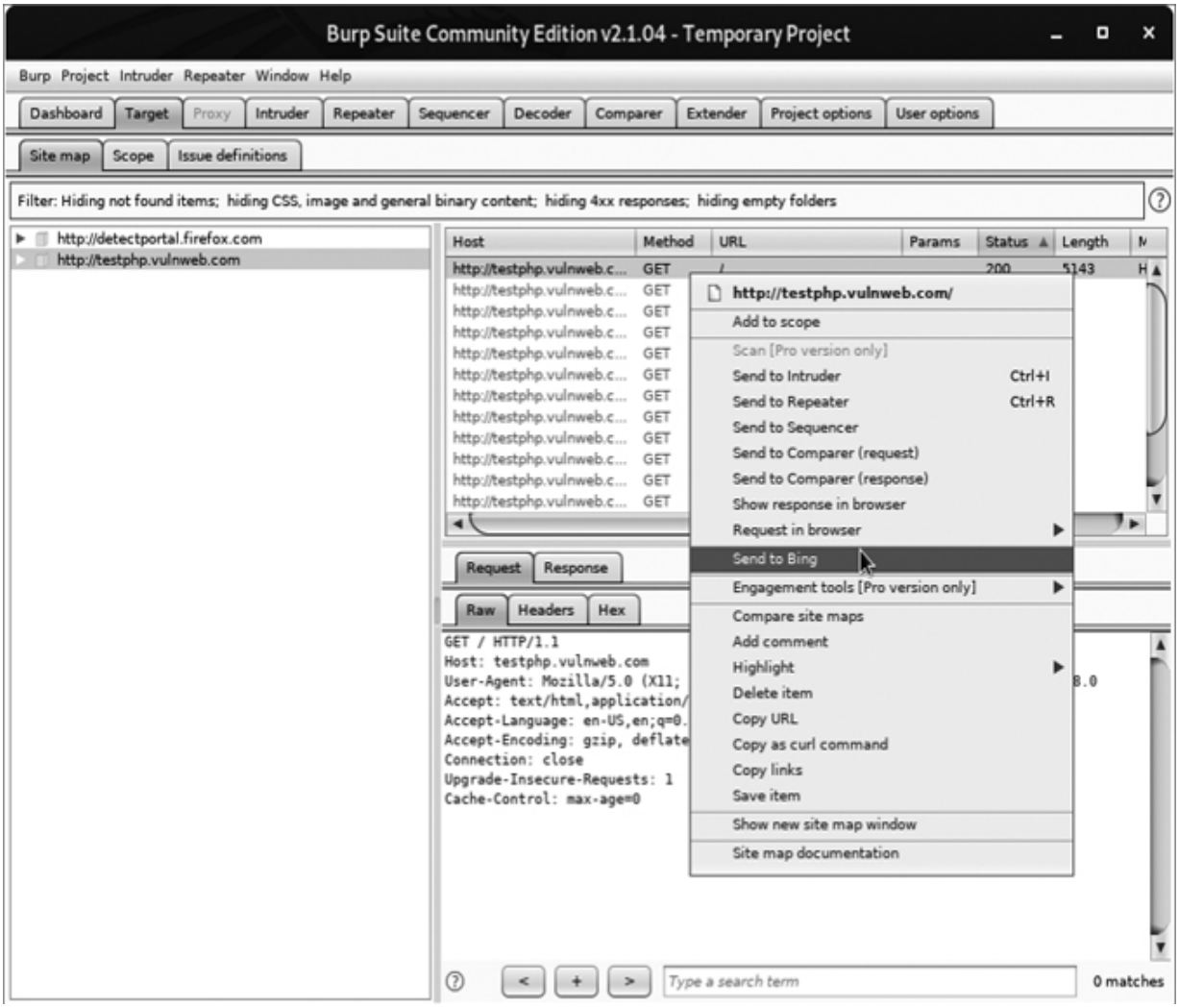


Figure 6-9: New menu option showing our extension

When you click this menu option, you should start to see results from Bing, as in *Figure 6-10*. The kind of result you get will depend on the output you chose when you loaded the extension.

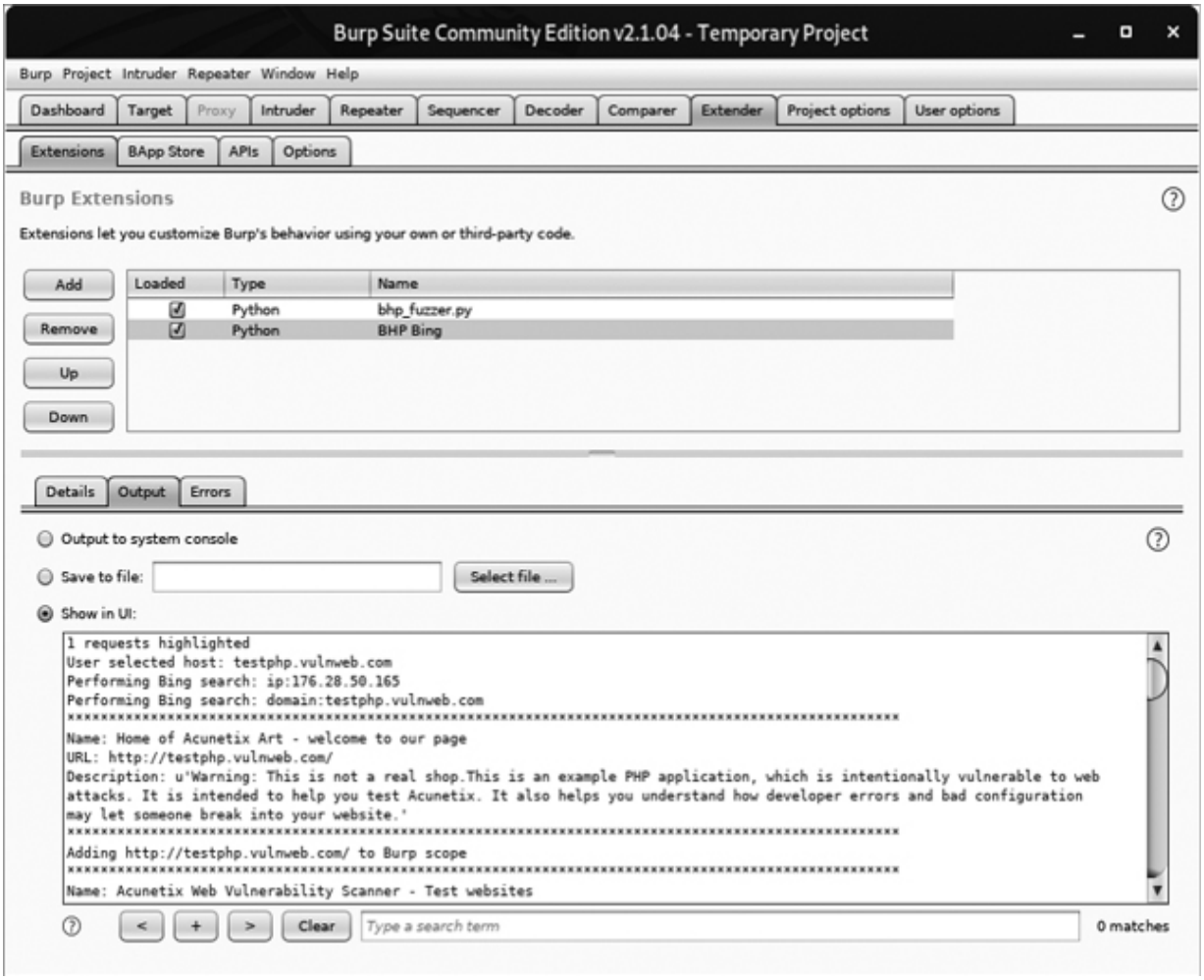


Figure 6-10: Our extension providing output from the Bing API search

If you click the **Target** tab in Burp and select **Scope**, you should see new items automatically added to the target scope, as shown in *Figure 6-11*. The target scope limits activities such as attacks, spidering, and scans to the defined hosts only.

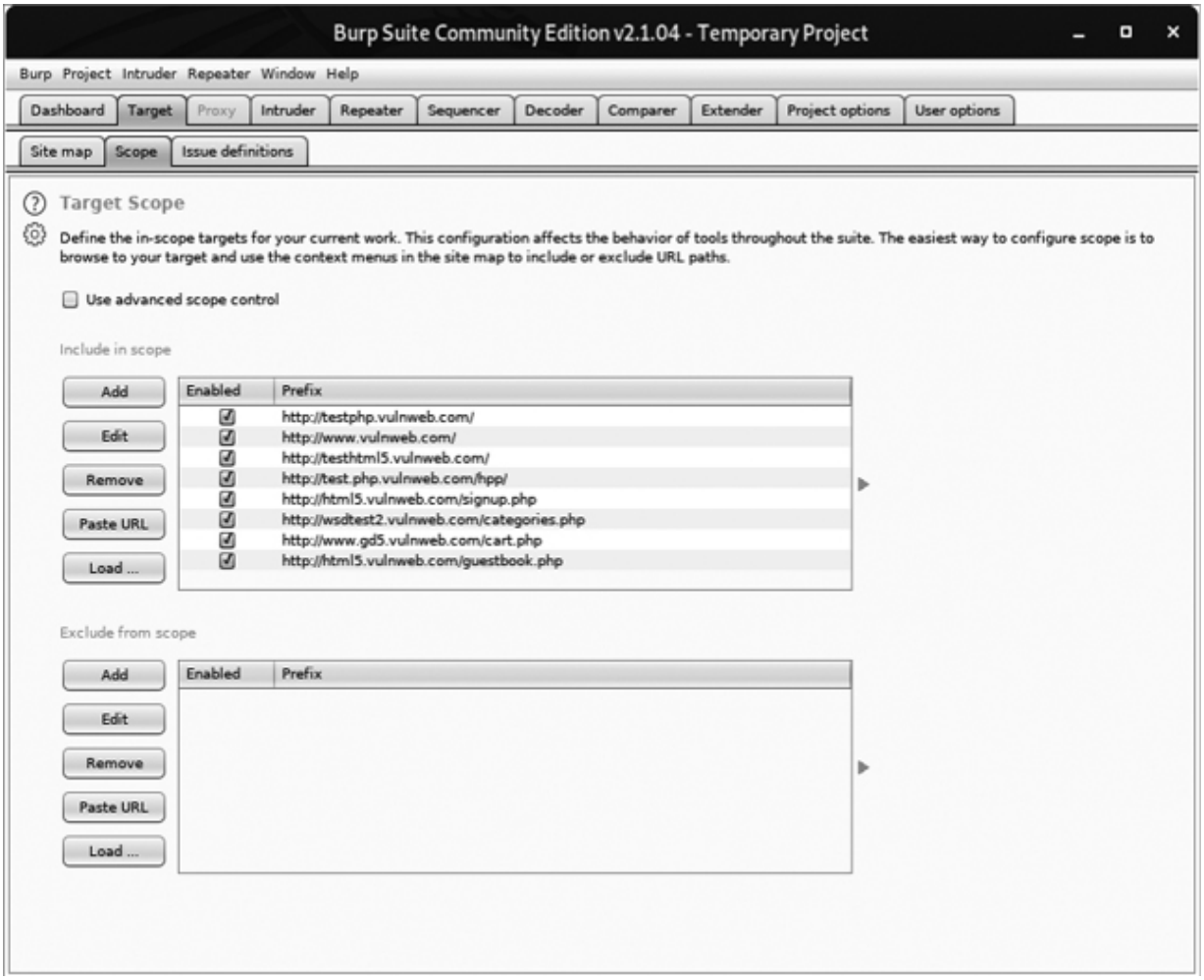


Figure 6-11: Discovered hosts are automatically added to Burp's target scope.

Turning Website Content into Password Gold

Many times, security comes down to one thing: user passwords. It's sad but true. Making things worse, when it comes to web applications, especially custom ones, it's all too common to discover that they don't lock users out of their accounts after a certain number of failed authentication attempts. In other instances, they don't enforce strong passwords. In these cases, an online password-guessing session like the one in the last chapter might be just the ticket to gain access to the site.

The trick to online password guessing is getting the right word list. You can't test 10 million passwords if you're in a hurry, so you need

to be able to create a word list targeted to the site in question. Of course, there are scripts in Kali Linux that crawl a website and generate a word list based on site content. But if you've already used Burp to scan the site, why send more traffic just to generate a word list? Plus, those scripts usually have a ton of command line arguments to remember. If you're anything like us, you've already memorized enough command line arguments to impress your friends, so let's make Burp do the heavy lifting.

Open *bhp_wordlist.py* and knock out this code:

```
from burp import IBurpExtender
from burp import IContextMenuFactory

from java.util import ArrayList
from javax.swing import JMenuItem

from datetime import datetime
from HTMLParser import HTMLParser

import re

class TagStripper(HTMLParser):
    def __init__(self):
        HTMLParser.__init__(self)
        self.page_text = []

    def handle_data(self, data):
     1 self.page_text.append(data)

    def handle_comment(self, data):
     2 self.page_text.append(data)

    def strip(self, html):
        self.feed(html)
     3 return " ".join(self.page_text)


class BurpExtender(IBurpExtender, IContextMenuFactory):
    def registerExtenderCallbacks(self, callbacks):
        self._callbacks = callbacks
        self._helpers   = callbacks.getHelpers()
        self.context    = None
        self.hosts      = set()
```

```
        # Start with something we know is common
    4 self.wordlist   = set(["password"])

        # we set up our extension
        callbacks.setExtensionName("BHP Wordlist")
        callbacks.registerContextMenuFactory(self)

        return

    def createMenuItems(self, context_menu):
        self.context = context_menu
        menu_list = ArrayList()
        menu_list.add(JMenuItem(
            "Create Wordlist",
actionPerformed=self.wordlist_menu))

        return menu_list
```

The code in this listing should be pretty familiar by now. We start by importing the required modules. A helper `TagStripper` class will allow us to strip the HTML tags out of the HTTP responses we process later on. Its `handle_data` method stores the page text 1 in a member variable. We also define the `handle_comment` method because we want to add the words stored in developer comments to the password list as well. Under the covers, `handle_comment` just calls `handle_data`2 (in case we want to change how we process page text down the road).

The `strip` method feeds HTML code to the base class, `HTMLParser`, and returns the resulting page text 3, which will come in handy later. The rest is almost exactly the same as the start of the *bhp_bing.py* script we just finished. Once again, the goal is to create a context menu item in the Burp UI. The only thing new here is that we store our word list in a `set`, which ensures that we don't introduce duplicate words as we go. We initialize the `set` with everyone's favorite password, `password`4, just to make sure it ends up in our final list.

Now let's add the logic to take the selected HTTP traffic from Burp and turn it into a base word list:

```
    def wordlist_menu(self,event):
        # grab the details of what the user clicked
        http_traffic = self.context.getSelectedMessages()

        for traffic in http_traffic:
            http_service  = traffic.getHttpService()
            host          = http_service.getHost()
         1  self.hosts.add(host)

            http_response = traffic.getResponse()
            if http_response:
             2  self.get_words(http_response)

    self.display_wordlist()
    return

    def get_words(self, http_response):
        headers, body =
http_response.tostring().split('\r\n\r\n', 1)

        # skip non-text responses
     3  if headers.lower().find("content-type: text") == -1:
            return

        tag_stripper = TagStripper()
     4  page_text = tag_stripper.strip(body)

     5  words = re.findall("[a-zA-Z]\w{2,}", page_text)

        for word in words:
            # filter out long strings
            if len(word) <= 12:
             6  self.wordlist.add(word.lower())

        return
```

Our first order of business is to define the `wordlist_menu` method, which handles menu clicks. It saves the name of the responding host 1 for later and then retrieves the HTTP response and feeds it to the `get_words` method 2. From there, `get_words` checks the response header to make sure we're processing text-based responses only 3. The `TagStripper` class 4 strips the HTML code from the rest of the page text. We use a regular expression to find all words starting with

an alphabetic character and two or more “word” characters as specified with the `\w{2,}` regular expression 5. We save the words that match this pattern to the `wordlist` in lowercase 6.

Now let’s polish the script by giving it the ability to mangle and display the captured word list:

```
    def mangle(self, word):
        year     = datetime.now().year
         suffixes = ["", "1", "!", year] 1
        mangled  = []

        for password in (word, word.capitalize()):
            for suffix in suffixes:
                 mangled.append("%s%s" % (password, suffix))
2

        return mangled

    def display_wordlist(self):
         print ("#!comment: BHP Wordlist for site(s) %s" % ",
".join(self.hosts)) 3

        for word in sorted(self.wordlist):
            for password in self.mangle(word):
                print password

        return
```

Very nice! The `mangle` method takes a base word and turns it into a number of password guesses based on some common password creation strategies. In this simple example, we create a list of suffixes to tack on the end of the base word, including the current year 1. Next, we loop through each suffix and add it to the base word 2 to create a unique password attempt. We do another loop with a capitalized version of the base word for good measure. In the `display_wordlist` method, we print a “John the Ripper”–style comment 3 to remind us which sites we used to generate this word list. Then we mangle each base word and print the results. Time to take this baby for a spin.

Kicking the Tires

Click the **Extender** tab in Burp, click the **Add** button, and then use the same procedure we used for our previous extensions to get the Wordlist extension working.

In the Dashboard tab, select **New live task**, as shown in *Figure 6-12*.

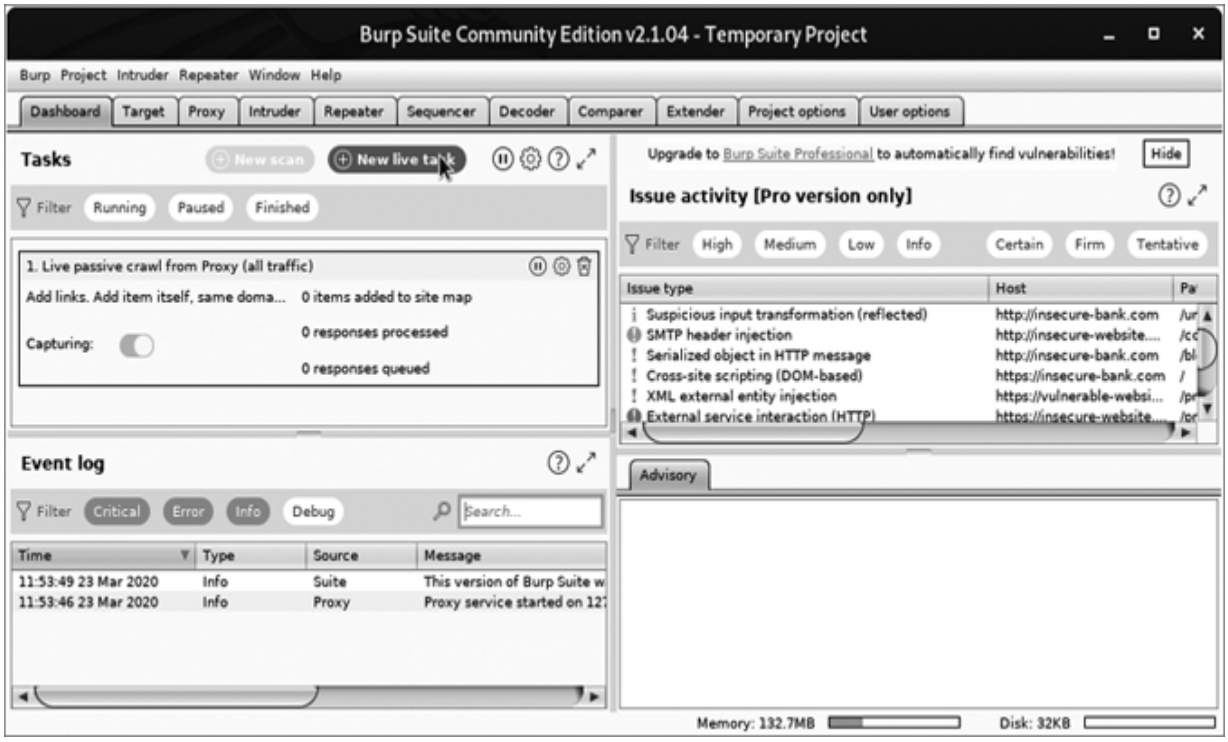


Figure 6-12: Starting a live passive scan with Burp

When the dialog appears, choose **Add all links observed in traffic**, as shown in *Figure 6-13*, and click **OK**.



Figure 6-13: Configuring the live passive scan with Burp

After you've configured the scan, browse to *http://testphp.vulnweb.com/* to run it. Once Burp has visited all the links on the target site, select all the requests in the top-right pane of the **Target** tab, right-click them to bring up the context menu, and select **Create Wordlist**, as shown in *Figure 6-14*.

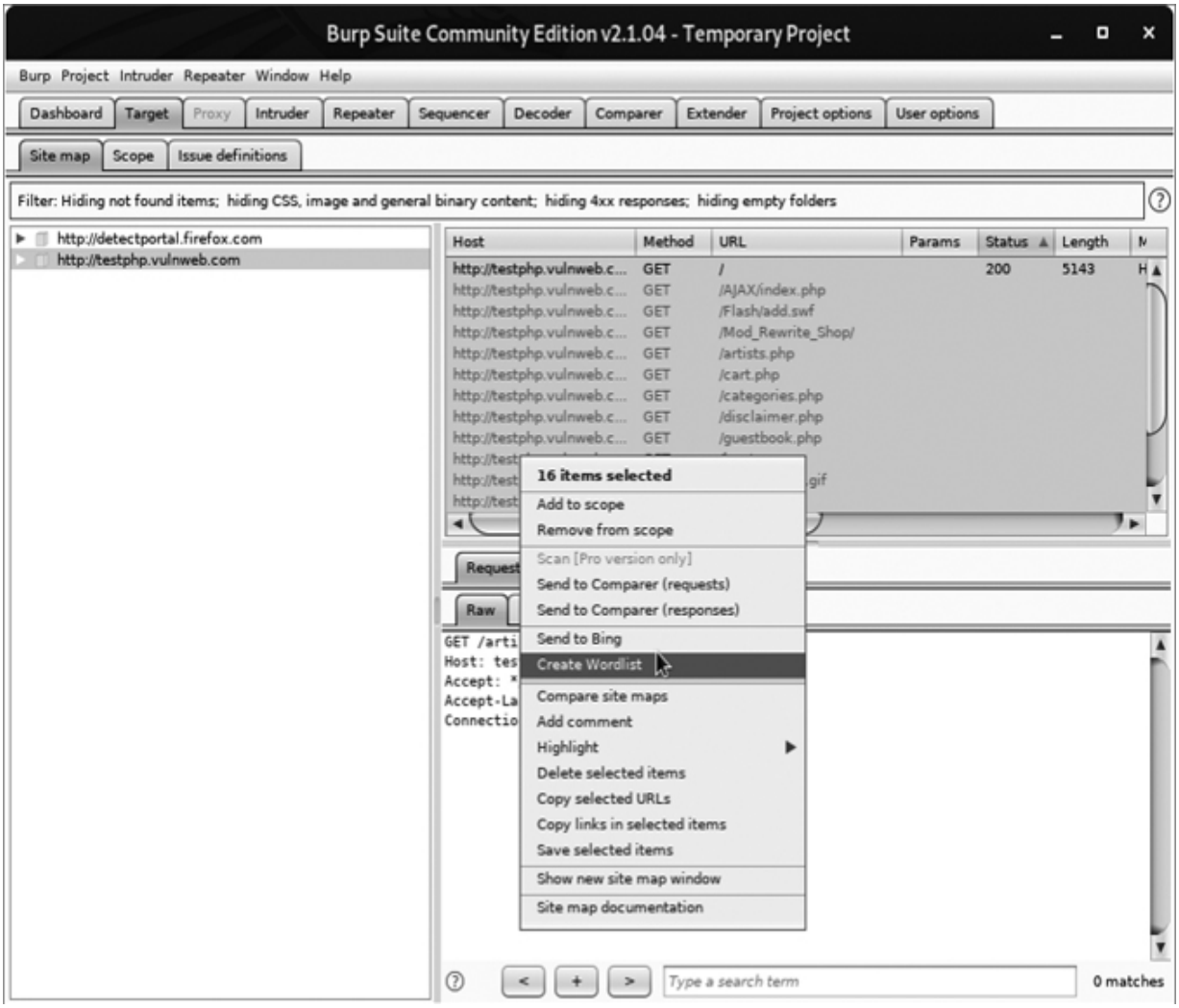


Figure 6-14: Sending the requests to the BHP Wordlist extension

Now check the Output tab of the extension. In practice, we'd save its output to a file, but for demonstration purposes we display the word list in Burp, as shown in *Figure 6-15*.

You can now feed this list back into Burp Intruder to perform the actual password-guessing attack.

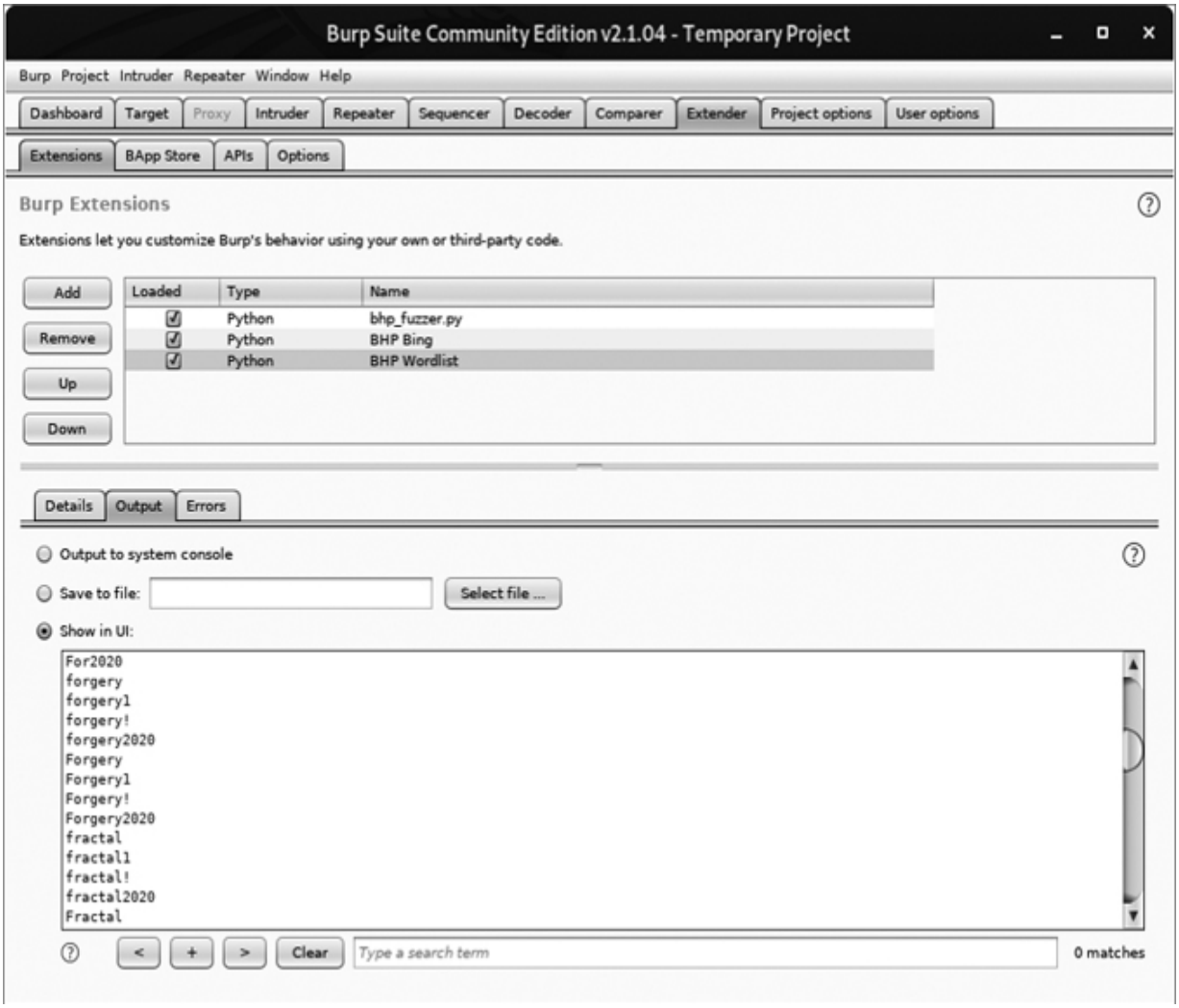


Figure 6-15: A password list based on content from the target website

We've now demonstrated a small subset of the Burp API by generating our own attack payloads, as well as building extensions that interact with the Burp UI. During a penetration test, you'll often encounter specific problems or automation needs, and the Burp Extender API provides an excellent interface to code your way out of a corner, or at least save you from having to continually copy and paste captured data from Burp to another tool.